

COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 4

5 Programming in C and C++ (djg11)

Pointers and
Memory
Management

- (a) A FIFO is implemented in C using a singly-linked list that maintains global head and tail pointers. These are initialised to represent the empty FIFO as follows:

```
struct fifo_entry *head_ptr = 0;
struct fifo_entry **tail_ptr = &head_ptr;
```

The entries in the FIFO use a union to store either an integer or a double-precision floating point number. A further field records which is stored. Give syntactically-accurate C code that defines `fifo_entry` and either a function `enqueue_int` or a function `enqueue_double` enqueueing a new value into the FIFO. [5 marks]

Answer:

```
struct fifo_entry
{ struct fifo_entry* next;
  union { int ival; double dval; } u;
  int is_int;
};
void enqueue_int(int newi) // only one of these need be provided
{
    struct fifo_entry *nv = (struct fifo_entry *)malloc(sizeof(struct fifo_entry));
    nv->is_int = 1; nv->u.ival = newi; nv->next = 0;
    *tail_ptr = nv;
    tail_ptr = &(nv->next);
}
void enqueue_double(double newd) // only one of these need be provided
{
    struct fifo_entry *nv = (struct fifo_entry *)malloc(sizeof(struct fifo_entry));
    nv->is_int = 0; nv->u.dval = newd; nv->next = 0;
    *tail_ptr = nv;
    tail_ptr = &(nv->next);
}
```

- (b) To avoid repetitive reallocation of memory, suppose now that FIFO entries that are no longer in use are to be saved in an auxiliary linked list. Give syntactically-accurate code that implements this approach and then discuss two other approaches for store management. [5 marks]

Answer: The spare entries can simply be chained using their `next` field (so answers involving more than a global variable, `struct fifo_entry *freelist;`, are inferior).

We modify the first line of `enqueue` to first check the freelist:

```
struct fifo_entry *nv;
if (freelist) { nv = freelist; freelist = freelist->next; }
else nv = (struct fifo_entry *)malloc(sizeof(struct fifo_entry));
```

and when an entry is no longer wanted we save it using

```
struct fifo_entry *ov = ...
ov -> next = freelist; freelist = ov;
```

One alternative approach is to directly return every unwanted entry to the heap using `free(ov)`. Heap management systems attract a (fairly small) overhead because they support requests that vary in size, whereas our own implementation above is faster, but it never ultimately returns memory for other uses. Another extreme approach is to ignore memory deallocation and assume there's enough swap to use it before the program finishes!

A key variation is to allocate heap store in arenas that consist of a large number of cells and implement our own allocator within the arena. The arena can be managed as an array owing to only having one entry type which is a fixed-size item. Keeping a linked list in an arena array should then be compared with a circular FIFO in an array. Using a very large arena containing a linked list with LIFO reuse does not sit on very much physical memory, just virtual memory space, which is abundant in 64-bit systems. But a very large circular FIFO does consume a lot of memory because space is not reused until all that was nominally allocated has been used once.

Garbage collection could also be mentioned, which will be useful if the lifetime of the de-queued cells was hard to determine, but since they here contain just one register's worth of POD data, there is no need to keep any handle on a dequeued cell.

Full credit will be awarded without all of these points being made. I would tend to use the initial approach and none of the alternatives.

C++ classes

- (c) The C code in part (a) suffers from a lack of encapsulation – variables like `head_ptr` are visible to the the rest of the program. Write C++ defining a `class FIFO` which maintains a single FIFO implemented using elements `fifo_entry` as defined in your answer above, but which only exports member functions `enqueue_int`, `enqueue_double`, `isempty` and

```
void dequeue(void do_I(int), void do_D(double));
```

It should not be possible to create an instance of `class FIFO` and storage allocation/deallocation should use C++ mechanisms rather than those of C. Your C++ is not required to be syntactically accurate, but should capture the main concepts. It is not necessary to give full code for the above four member functions – focusing on allocation and deallocation of `fifo_entry` elements suffices. [Note: `do_I` and `do_D` are user-provided processing functions to be applied to the dequeued value.] [6 marks]

Answer: One possibility (compilable, exact C++ syntax was not required in exam answers):

```
class FIFO {
private:
    // C++ static member initialisations (legally) occur out-of-class:
    static struct fifo_entry *head_ptr; /* = 0; */
    static struct fifo_entry **tail_ptr; /* = &head_ptr; */
    FIFO() {} // private constructor to forbid instances
public:
    static void enqueue_int(int); // see below
    static void enqueue_double(double); // see below
    static bool isempty() { return (!head_ptr); }
    static void dequeue(void do_I(int), void do_D(double)) {
```

```

    if (!head_ptr) return;
    struct fifo_entry *nv = head_ptr;
    if (tail_ptr == &nv->next) tail_ptr = &head_ptr;
    head_ptr = nv->next;
    if (nv->is_int) do_I(nv->u.ivalue); else do_D(nv->u.dvalue);
    delete nv;
}
};
struct fifo_entry *FIFO::head_ptr = 0;
struct fifo_entry **FIFO::tail_ptr = &FIFO::head_ptr;

```

Remark: the most important words in the question are: “use C++ mechanisms for allocation/deallocation” and “not be possible to create an instance of `class FIFO`”.

Mark allocation: one mark each for using `new` and `delete` correctly and another for making `head_ptr` and `tail_ptr` private. Another mark for noting the need for private (or deleted) constructor(s). Two more marks for declaring `static` member variables `head_ptr` and `tail_ptr` and for using `static` for the member functions `enqueue_int` etc. (as there are no instances of `class FIFO`).

Remark: here `class FIFO` is only used for encapsulation; one student used design pattern ‘singleton class’ which was awarded most marks even though it still allows one instance of `FIFO` to be constructed.

Storage is allocated in C++ by replacing the the first line of the C versions (above) of `enqueue` with

```
struct fifo_entry *nv = new struct fifo_entry;
```

Note that parameterless `new` can be used to allocate C-style `struct fifo_entry` (as a default constructor, which does nothing, will be created). Defining an explicit constructor for `struct fifo_entry` is unhelpful, but not penalised.

Deallocation requires the `delete` operator, destructors are not useful here. See `dequeue` above.

- template (d) The following lines approximate (e.g. omitting access qualifiers) analogous generic/templated class definitions in Java and C++. Explain which types X are valid for use in `Gen<X>` and `Tem<X>`.

```

[Java]: class      Gen<T>                { T v; Gen() { v = 1; } };
[C++]: template<typename T> class Tem { T v; Tem() { v = 1; } };

```

[4 marks]

Answer: In Java `Gen<X>` requires X to be a reference type, not a primitive type. But the constructor `Gen()` then does not compile as the assignment `v = 1` is type-incorrect. [Java templates work by type erasure – occurrences of `T` in the definition of `class Gen<T>` are effectively replaced by `Object` and appropriate casts inserted and there are no implicit casts from `int` to `Object`.]

In C++ `Tem<X>` operates effectively as macro replacement, so in principle all types X are possible. However the constructor `Tem()` requires that variables `v` of type X support the assignment `v=1`;. This is possible for many primitive types, such as `long`, `bool` etc. Additionally X may be a user-defined `class` which has a constructor taking an `int`.

[Subtlety not required here for full marks: alternatively X could be a class for which the `operator=` has been overloaded to implement the assignment `v=1`;.]
